

Лабораторная работа №1. Вычисление числа π с использованием OpenMP

Студент: Шамсутдинов Ренат Русланович Группа: 11-401

1. Описание работы

Реализованы последовательная и параллельная (`#pragma omp parallel for reduction(+:sum)`) версии вычисления π по ряду Лейбница, время измерено через `omp_get_wtime()`. Проведены две серии экспериментов: зависимость от числа потоков ($n = 10^9$) и от размера задачи (8 потоков). Условия: Windows, GCC 16.1.0 (MSYS2 UCRT64), gcc -O2 -fopenmp pi.c -o pi -lm; процессор AMD Ryzen 7 5700U (8 ядер, 16 потоков). Ускорение = $t_{\text{посл}} / t_{\text{OpenMP}}$; каждая конфигурация запускалась один раз.

2. Эксперимент 1. Влияние количества потоков ($n = 10^9$)

Потоки	Последовательно, с	OpenMP, с	Ускорение
1	2.5690	2.5750	1.00
2	2.5690	1.3330	1.93
4	2.5690	0.7250	3.54
8	2.5690	0.4920	5.22
16	2.5690	0.3890	6.60

3. Эксперимент 2. Влияние размера задачи (8 потоков)

n	Последовательно, с	OpenMP (8 потоков), с	Ускорение
10^7	0.0260	0.0070	3.71
10^8	0.2550	0.0520	4.90
10^9	2.5670	0.4270	6.01
10^{10}	25.5470	3.8010	6.72

4. Вывод

С ростом числа потоков время при $n = 10^9$ монотонно уменьшалось: с 2,58 с на одном потоке до 0,39 с на 16 потоках, ускорение выросло с 1,00 до 6,60. До 4 потоков рост близок к линейному: ускорение 1,93 на 2 потоках (эффективность около 97%) и 3,54 на 4 потоках (около 88%). Заметно расти ускорение перестало после 8 потоков: удвоение с 8 до 16 потоков дало лишь 5,22 $\rightarrow$ 6,60 (около +26%), поскольку у процессора 8 физических ядер, а дополнительные логические потоки (SMT) делят их ресурсы. Ускорение всегда ниже числа потоков, так как есть расходы на создание потоков и редукцию, а при полной загрузке процессора (особенно ноутбука) частота обычно снижается. Размер задачи заметно влияет на эффективность: при $n = 10^7$ ускорение лишь 3,71 (параллельный счёт занимает 7 мс, и фиксированные расходы на потоки составляют большую долю), а с ростом n оно увеличивается до 4,90, 6,01 и 6,72 при $n = 10^{10}$. Для больших n накладные расходы становятся пренебрежимо малы, и ускорение приближается к пределу, заданному числом физических ядер. Следует учитывать разброс однократных замеров: 8 потоков при $n = 10^9$ дали ускорение 5,22 в первом эксперименте и 6,01 во втором. Итого: чем больше задача, тем лучше окупается распараллеливание, но выигрыш ограничен числом физических ядер. Абсолютная ошибка убывает как $\approx 1/n$ (от 10^{-7} до 10^{-10}), поэтому ряд Лейбница сходится медленно и точность растёт только с увеличением n .